

Human-in-the-Loop Adaptive Alert Noise Reduction System for Backend Reliability Monitoring

Project Proposal and Business Case

Document Information	
Author(s):	Dixin Zhang
Keywords:	Backend reliability; alert fatigue; human-in-the-loop feedback; adaptive alerting; anomaly detection; alert prioritization; suppression; deduplication

Document Revision History

Version	Date	Author	Changes Made
1.0	6/29/2026	Dixin Zhang	Completed initial proposal and business case draft.
1.1	7/6/2026	Dixin Zhang	Added Architecture Overview, Evaluation Design, and refined the alert ranking method description.

Reviewers

Name	Contact Information
<Instructor Name>	Canvas

Table of Contents

Introduction	3
Business Problem and/or Opportunity	3
Project Outcomes.	4
Success Criteria (including measurement metrics)	4
Scope	5
Architecture Overview	6
Evaluation Design	6
End-User and Admin Features	6
Out of Scope	7
End State	7
Assumptions	7
Known Risks	7
Appendices	8

Introduction

The objective of this project is a human-in-the-loop adaptive alerting system for backend reliability monitoring. The system focuses on backend jobs, scheduled workflows, batch pipelines, queue-based workers, and other recurring execution processes where reliability problems manifest as latency spikes, retries, partial failures, queue delays or repeated low-value alerts.

The project aims to move beyond a static threshold-only alerting model and propose a system that combines baseline alert generation, unsupervised alert suppression rules, alert deduplication, adaptive priority ranking driven by user-provided feedback, and intuitive explainer recommendation. In the system, operational users can mark alerts as useful, noisy, duplicate, expected or false positive, and administrators can see how that feedback informs the system's thresholds, suppression rules and alert priority over time.

The research question for this project is: can a human-feedback-driven adaptive alerting system reduce the number of false positive alerts and duplicate alerts while maintaining or increasing the detection of real reliability incidents? The project will benchmark the performance of the proposed approach against rule-based alert suppression and fixed threshold alerting.

Business Problem and/or Opportunity

Modern software teams depend on observability tools to detect production failures quickly. However, backend environments often produce large volumes of alerts that are not equally important. Some alerts represent real incidents, while others are false positives, duplicates, expected maintenance behavior, transient retries, or low-severity noise. When engineers receive too many low-value alerts, they may become slower to respond, ignore notifications, or miss the signal that actually matters.

Industry evidence shows that alert fatigue is a real operational problem, not only a dashboard design issue. Reporting on Grafana Labs' 2026 observability findings noted that 30% of workers cited alert fatigue as the biggest obstacle to faster incident response. Reporting on Splunk's State of Observability 2025 also noted that roughly three-quarters of UK IT teams experienced outages related to missed, ignored, or suppressed alerts. These findings support the business need for alert systems that reduce noise while keeping important incidents visible.

The business opportunity is to build a focused reliability platform for backend jobs and workflows. Instead of trying to replace full observability platforms such as Grafana, Splunk, or OpenSearch, this project narrows the scope to the alert quality problem: which backend alerts deserve attention, which can be grouped or suppressed, and how human feedback can improve the next round of alert decisions.

Project Outcomes.

The expected outcome is a working web-based prototype that demonstrates an adaptive alert noise reduction system for backend reliability monitoring. The final system will include both end-user workflows and administrator workflows.

- A backend metrics and alert ingestion layer that accepts simulated or Prometheus-style signals such as job latency, failure rate, retry count, timeout count, queue delay, and partial-success status.
- A baseline alert engine that supports fixed thresholds and simple rules so the proposed method can be evaluated against traditional approaches.
- A feedback module where users can classify alerts as useful, noisy, duplicate, expected, false positive, or real incident.
- An adaptive scoring engine that adjusts alert priority based on historical behavior, user feedback, job category, service criticality, time window, and repeated failure patterns.
- Suppression and deduplication features that group repeated alerts and reduce redundant notifications by job, service, failure category, and time window.
- An explainability panel that shows why an alert was raised, lowered, grouped, or suppressed.
- An administrative console for configuring monitoring objects, thresholds, suppression rules, feedback weights, users, roles, and model recommendations.
- An evaluation dashboard that compares the proposed method with fixed-threshold and rule-based baselines using reliability and alert-quality metrics.

Success Criteria (including measurement metrics)

Success will be measured by both system deliverables and experimental results. The project does not need to prove that all alert fatigue can be eliminated. It should demonstrate that a feedback-driven adaptive approach can reduce alert noise while maintaining useful incident detection.

Metric	Target or Measurement Approach	Why It Matters
False Positive Rate	Reduce false positives by at least 25% compared with fixed threshold alerting.	Shows whether noise is actually reduced.
Duplicate Alert Rate	Reduce duplicate alerts by at least 30% through grouping and deduplication.	Measures alert consolidation effectiveness.
Alert Actionability Rate	Increase the share of alerts marked useful or real incident by at least 20%.	Measures whether engineers see higher-value alerts.

	Page 4 of 8	

Precision and Recall	Compare precision and recall across fixed threshold, rule suppression, and adaptive scoring.	Ensures noise reduction does not hide real incidents.
Mean Time to Detect	Measure whether real anomalies are detected as fast as or faster than the baseline.	Connects alert quality to reliability response.
Mean Time to Acknowledge	Measure whether prioritized alerts are acknowledged faster in the prototype workflow.	Shows operational usability.
Feedback Acceptance Rate	Track how often users agree with adaptive recommendations after seeing explanations.	Measures trust in the feedback loop.
Admin-approved Recommendation Rate	Track how many suggested threshold or suppression changes are approved by an admin.	Prevents unsafe automatic changes.

Scope

The project scope is intentionally focused on backend jobs and workflow reliability rather than full-stack observability. The following features will be delivered as part of the capstone prototype:

- Frontend: React-based dashboard for alert review, feedback submission, trend viewing, explainability, and admin configuration.
- Backend: Python FastAPI or Flask service for alert ingestion, feedback capture, scoring, suppression, deduplication, and recommendation APIs.
- Database: PostgreSQL schema for jobs, services, alert events, feedback labels, suppression rules, threshold versions, users, roles, and audit logs.
- Processing layer: Celery workers or background jobs for scoring alerts, aggregating duplicate alerts, and recalculating adaptive thresholds.
- Metrics source: simulated backend reliability data and Prometheus-style metric inputs for latency, failure rate, retries, queue delay, and throughput.
- Scoring approach: rule filtering first, then statistical anomaly detection, then feedback-driven weight adjustment and priority ranking.
- Evaluation: controlled experiments comparing fixed thresholds, rule suppression, and the proposed adaptive feedback method.
- Deployment: Docker-based local deployment with sample data and reproducible evaluation scripts.

Architecture Overview

The system will follow a three-tier web architecture with a React frontend, a Python FastAPI backend, and a PostgreSQL data layer. End users and administrators will access the web dashboard to review alerts, submit feedback, inspect explanations, and manage configuration. The backend API will ingest backend job metrics, generate baseline alerts, apply suppression and deduplication logic, recalculate adaptive priority scores, and expose recommendation endpoints. PostgreSQL will persist jobs, services, alert events, feedback labels, suppression rules, threshold versions, user roles, and audit logs. Background workers will process asynchronous tasks such as duplicate grouping, score recalculation, and threshold recommendation generation. This architecture separates presentation, decision logic, and storage concerns while supporting controlled evaluation of alternative alerting strategies.

Evaluation Design

Evaluation will be conducted using controlled experiments on simulated backend workload data with injected anomalies and labeled alert outcomes. The dataset will include normal runs and incident scenarios such as latency spikes, failure bursts, retry storms, queue backlogs, and timeout clusters. Each scenario will be replayed through three methods: fixed-threshold alerting, rule-based suppression, and the proposed adaptive feedback-driven method. A ground-truth incident log will be created during simulation so that each generated alert can be mapped to a known incident, duplicate event, or false positive.

User feedback labels will be collected through the prototype workflow or pre-seeded from the simulated evaluation dataset to represent useful, noisy, duplicate, expected, false positive, and real incident outcomes. The three methods will be compared using false positive rate, duplicate alert rate, precision, recall, mean time to detect, mean time to acknowledge, feedback acceptance rate, and admin-approved recommendation rate. This design will allow the project to measure whether adaptive prioritization reduces noise without suppressing meaningful reliability incidents.

End-User and Admin Features

End users include developers, on-call engineers, and reliability analysts. They need to view current backend alerts, understand why an alert matters, give feedback quickly, and review trends for recurring job failures. Their core features will include alert list, alert detail view, feedback actions, duplicate grouping, job-level history, and explanation notes.

Admin users include SREs, platform administrators, and engineering managers. They need to configure monitoring objects, thresholds, suppression rules, feedback weights, and approval workflows. Their core features will include user and role management, rule configuration, recommendation review, audit logs, and an evaluation dashboard showing alert quality over time.

Out of Scope

The following items will not be delivered in the first version of this project and may be considered future enhancements:

- A complete replacement for enterprise observability platforms such as Grafana, Splunk, Datadog, or OpenSearch.
- Production-scale ingestion from many real cloud accounts, clusters, or microservice environments.
- Autonomous incident remediation, automatic rollback, or automatic production configuration changes without admin approval.
- Heavy deep-learning models that require large training datasets or specialized GPU infrastructure.
- A general security operations center alert platform; this prototype focuses on backend reliability alerts, not all cybersecurity alerts.
- Mobile applications, paging integrations, or full enterprise identity integrations beyond a simple role-based prototype.

End State

At the end of the project, the deliverable will be a deployable capstone prototype with documented architecture, source code, sample data, and an evaluation report. A user will be able to ingest backend job signals, view generated alerts, provide feedback, see alerts grouped or reprioritized, and compare the adaptive method against baseline alerting approaches.

Assumptions

- Backend job metrics can be simulated or imported in a Prometheus-style format for the purpose of the prototype.
- Users can provide meaningful feedback labels such as useful, noisy, duplicate, expected, false positive, or real incident.
- Backend jobs can be categorized by service, workflow, severity, ownership, and business criticality.
- Historical alert data will be large enough to support basic statistical scoring and before/after evaluation.
- The project timeline supports a prototype with controlled evaluation, not a production-ready enterprise observability product.
- Administrative approval will be required before adaptive threshold or suppression recommendations become active.

Known Risks

Risk		Potential Impact	Mitigation
		Page 7 of 8	

Insufficient realistic data	Evaluation may not represent real incident behavior.	Use multiple simulated workloads, inject known failures, and clearly state data limitations.
Over-suppression of important alerts	A real reliability issue could be hidden by aggressive noise reduction.	Keep recall as a required metric and require admin approval for suppression changes.
Feedback bias	The system may learn from inconsistent or low-quality user labels.	Track feedback source, confidence, and disagreement; use explanations and audit logs.
Scope creep	Trying to build a full observability platform could make the project too large.	Limit scope to backend jobs, alert quality, and evaluation against two baselines.
Explainability complexity	Users may not trust adaptive scoring if the reasons are unclear.	Provide simple explanation fields such as recent false-positive rate, latency drift, and service criticality.
Integration difficulty	Connecting to real external observability tools may take more time than expected.	Use simulated and Prometheus-style data as the main input; treat external integrations as optional future work.

Appendices

Appendix A: Proposed Technical Stack

- Frontend: React with Chart.js or ECharts for alert trends, evaluation charts, and admin dashboards.
- Backend API: Python FastAPI or Flask.
- Database: PostgreSQL for persistent alerts, feedback, rules, recommendations, and audit data.
- Background Processing: Celery or lightweight background workers for scoring and aggregation tasks.
- Metric Input: simulated backend job data and Prometheus-style metrics.
- Deployment: Docker Compose for local reproducibility.